第1章

倒排索引

Inverted Index

提纲

① 信息检索概述

② 倒排索引

③ 布尔查询的处理

提纲

① **信息检索概述**

② **倒排索引**

③ **布尔查询的处理**

信息检索 Information Retrieval

- Information Retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).

信息检索 Information Retrieval

- **Information Retrieval (IR)** is finding material (usually **documents**) of an unstructured nature (usually **text**) that satisfies an information need from within large **collections** (usually stored on computers).

信息检索 Information Retrieval

- **信息检索**是从大规模非结构化数据（通常是**文本**）
- 的**集合**（通常保存在计算机上）中
- 找出满足用户信息需求的资料（通常是**文档**）
- 的过程

信息检索 Information Retrieval

- **Document – 文档**
  - **检索系统的检索对象**
- **Unstructured – 非结构化**
  - **没有清晰和明显的语义结构的数据，计算机不易处理**
- **Information need – 信息需求**
- **Collection – 文档集、语料库**

IR vs 数据库: 结构化 vs 非结构化数据

- **结构化数据即指“表”中的数据**

(雇员) (管理者，老板) (收入，薪水)

Employee	Manager	Salary
张三	Jones	50000
李四	Smith	60000
王五	Smith	50000

数据库常常支持范围或者精确匹配查询 。e.g.,
Salary < 60000 AND Manager = Smith.
Employee = 李四

非结构化数据

- **通常指自由文本**
- 允许
  - 关键词加上操作符号的查询
  - 更复杂的概念性查询
    - 找出所有的有关药物滥用(drug abuse)的网页
- 经典的检索模型一般都针对**自由文本**进行处理
- **chatGPT- prompt**

半结构化数据

- **没有数据是完全无结构的**
  - <title>成都信息工程大学主页</title>
  - <body>…</body> …
- **半结构化查询**
  - Title contains data AND Bullets contain the search
  - （标题包含数据，项目符号包含搜索）

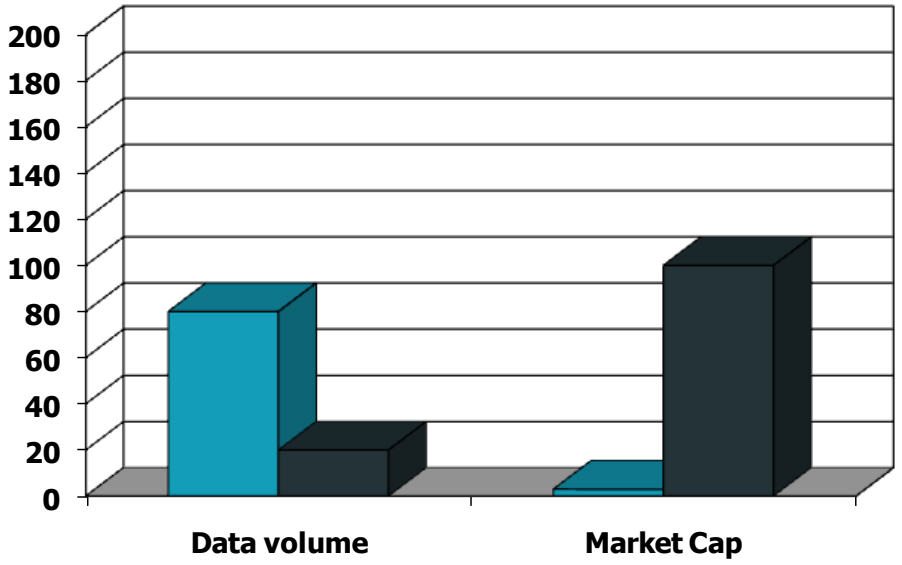
非结构化 vs. 结构化 vs. 半结构化

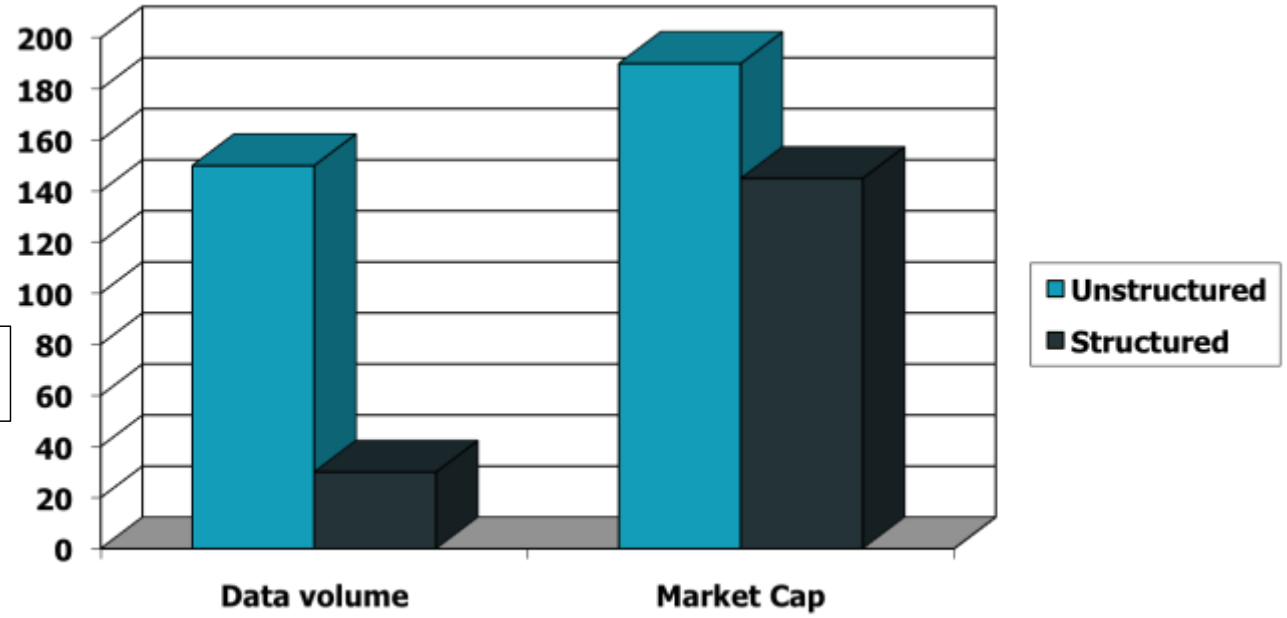
- 半结构化(Semi-structured)：
- <title>成都信息工程大学主页</title>
- <body>…</body> …

传统信息检索 vs. 现代信息检索

- **传统信息检索主要关注非结构化、半结构化数据**
- **现代信息检索中也处理结构化数据**

非结构化数据(文本) vs. 结构化数据(数据库) @ 1996年vs 2009年





数据量

市场规模





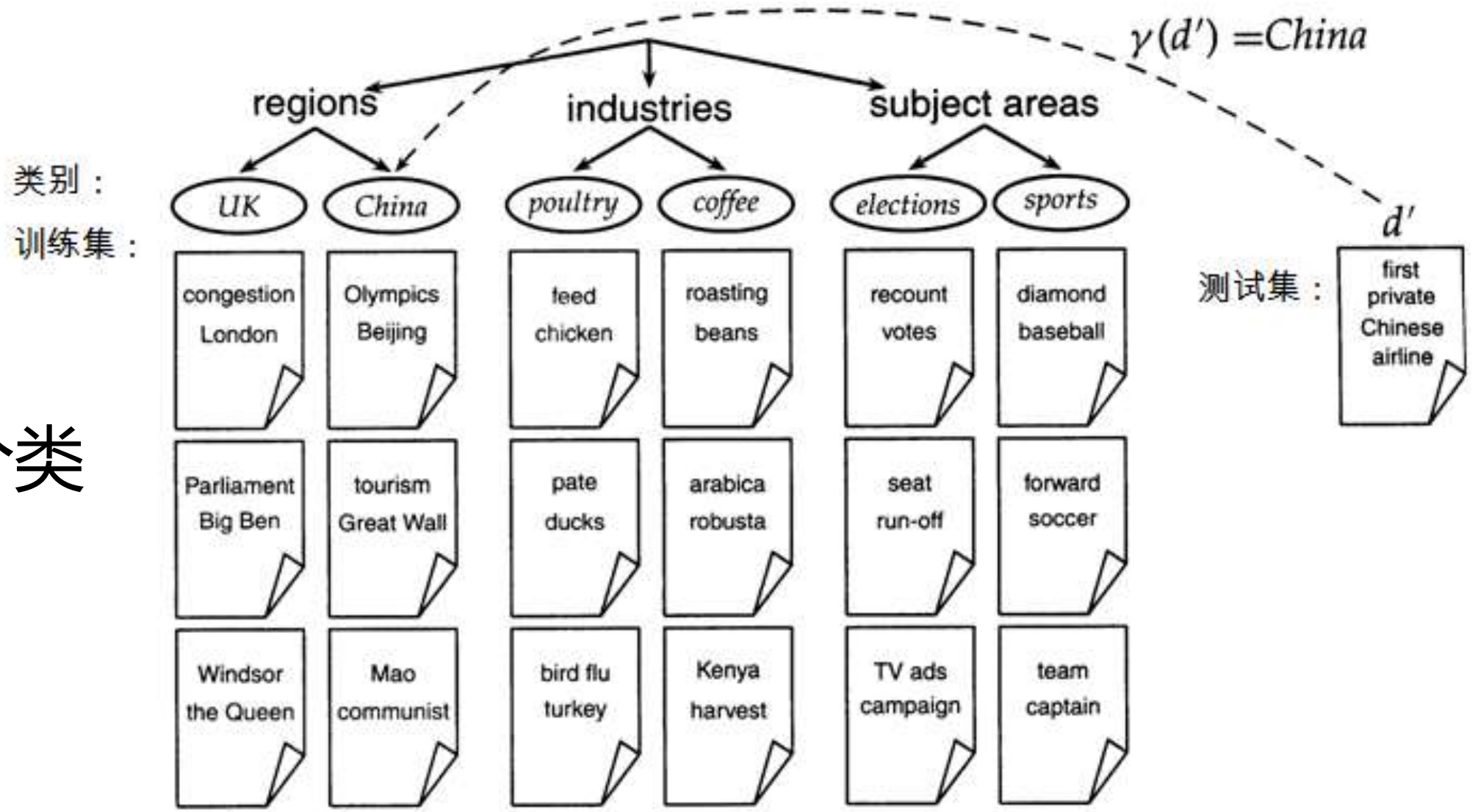




文档的进一步处理

- **聚类（clustering）**
  - **基于文档的内容进行自动聚团的任务**
- **分类（classification）**
  - **根据给定主题、固定的信息需求或者其他类别体系，将每篇文档分到一个或者多个类别的任务**
  - **通常做法：手工标注一些文档的类别，借助已标注文档来自动判断其他文档类别**

主题分类


$\gamma(d') = China$
regions
industries
subject areas
类别：
训练集：
UK
China
poultry
coffee
elections
sports
congestion
London
Olympics
Beijing
feed
chicken
roasting
beans
recount
votes
diamond
baseball
Parliament
Big Ben
tourism
Great Wall
pate
ducks
arabica
robusta
seat
run-off
forward
soccer
Windsor
the Queen
Mao
communist
bird flu
turkey
Kenya
harvest
TV ads
campaign
team
captain
d'
测试集：
first
private
Chinese
airline

信息检索的规模区分

- **大规模**
  - **Web搜索（web search）**
  - **要处理存储在数百万台计算机上的数十亿篇文档**
- **小规模**
  - **个人信息检索**
- **中等规模**
  - **企业、机构和特定领域的搜索**
  - **如公司内部文档、专利库或特定领域文献的搜索**
  - **文档往往存储在集中的文件系统中，由一台或者多台计算机提供搜索服务**

软件开发者更多面对的是个人搜索和企业级搜索这两类场景

提纲

① 信息检索概述

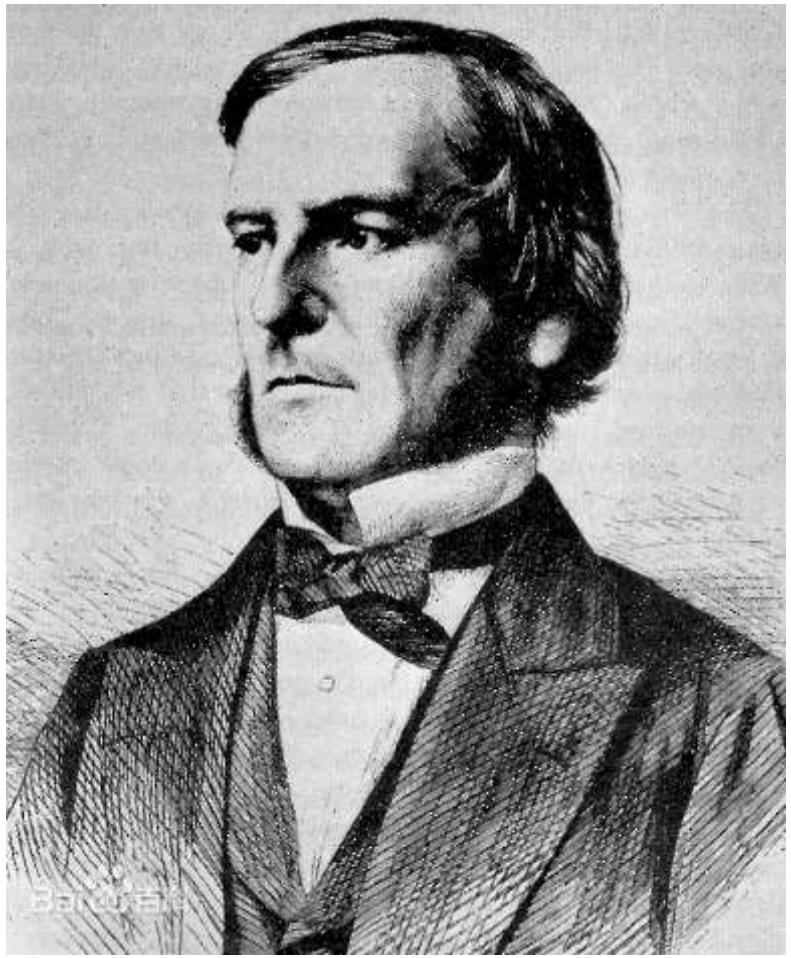
② **倒排索引**

③ 布尔查询的处理

布尔检索

- **逻辑运算**
- **针对布尔查询的检索**
- **布尔查询是指利用 AND, OR 或者 NOT操作符将词项连接起来的查询**

- **乔治•布尔**（George Boole，1815～1864），生于英格兰
- 19世纪最重要的数学家之一
- 出版了《逻辑的数学分析》，这是他对符号逻辑诸多贡献中的第一次
- 《思维规律的研究》为其最著名著作，在这本书中布尔介绍了现在以他的名字命名的**布尔代数**
- **由于其在符号逻辑运算中的特殊贡献，很多计算机语言中将逻辑运算称为布尔运算，将其结果称为布尔值**



布尔检索

- **针对布尔查询的检索，布尔查询是指利用 AND, OR 或者 NOT操作符将词项连接起来的查询**
  - 信息 AND 检索
  - 信息 OR 检索
  - 信息 AND 检索 AND NOT 教材
- Google的高级搜索?
- Baidu?

一个简单的例子(金庸小说)



- 金庸的哪部小说包含郭靖及周伯通
- 但是不包含金轮法王?
- 布尔表达式为:
- **郭靖 AND 周伯通 AND NOT 金轮法王**
- 笨方法： 从头到尾扫描所有剧本，对每部剧本判断它是否包含郭靖 AND 周伯通，同时又不包含金轮法王

词项-文档(term-doc)的关联矩阵

	倚天屠龙记	神雕侠侣	笑傲江湖	天龙八部	射雕英雄传	鹿鼎记
郭靖	0	1	0	0	1	0
黄蓉	0	1	0	0	1	0
郭襄	1	1	0	1	1	0
杨过	1	1	0	1	1	0
张无忌	1	0	0	0	0	0
周伯通	0	1	1	1	1	0
金轮法王	0	0	0	1	1	0
令狐冲	0	0	1	0	0	0
韦小宝	0	0	0	0	0	1

郭靖 AND 周伯通 AND NOT 金轮法王

矩阵实现：若某本小说包含某人物，则该位置的值为1，否则为0

关联向量(incidence vectors)

- 关联矩阵的每一列都是 0/1向量，每个0/1都对应一个词项
- 给定查询 **郭靖 AND 周伯通 AND NOT 金轮法王**
- 取出三个列向量，并对**金轮法王**的列向量求反，最后按位进行与操作
- **金轮法王：**000110 反码：111001
- 010010 AND 011110 AND 111001 = 010000

一个简单的例子(《莎士比亚全集》)

一个简单的例子(《莎士比亚全集》)

- 背景人物介绍
  - **Brutus**
  - **Caesar**
  - **Calpurnia Pisonis**
  - **Antony**
  - **Cleopetra**

一个简单的例子(《莎士比亚全集》)

背景人物介绍

- Brutus：即Marcus Brutus (马克斯·布鲁图斯)，他是晚期罗马共和国的一名元老院议员，组织并参与了对凯撒的谋杀
- Caesar：即Julius Caesar (尤利乌斯·凯撒)，也即凯撒大帝，罗马帝国的奠基者
- Calpurnia Pisonis (卡尔普尼亚·索尼斯)，凯撒的妻子

一个简单的例子(《莎士比亚全集》)

- Antony：马克·安东尼，古罗马的政治家和军事家
- Cleopetra： 克莉奥帕特拉七世(*Cleopatra* VII，公元前69—前30)，埃及托勒密王朝最后一位女王。即著名的埃及艳后

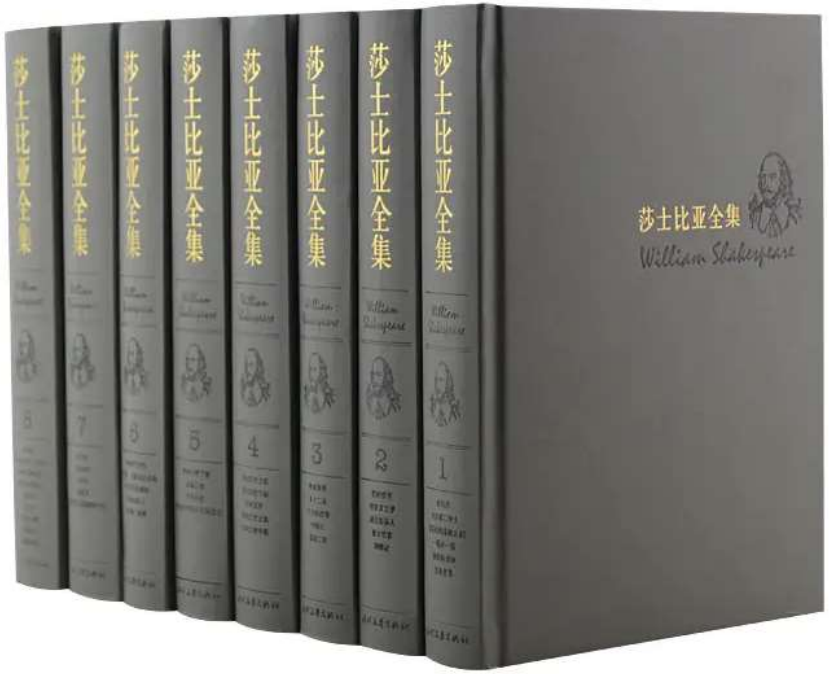
一个简单的例子(《莎士比亚全集》)

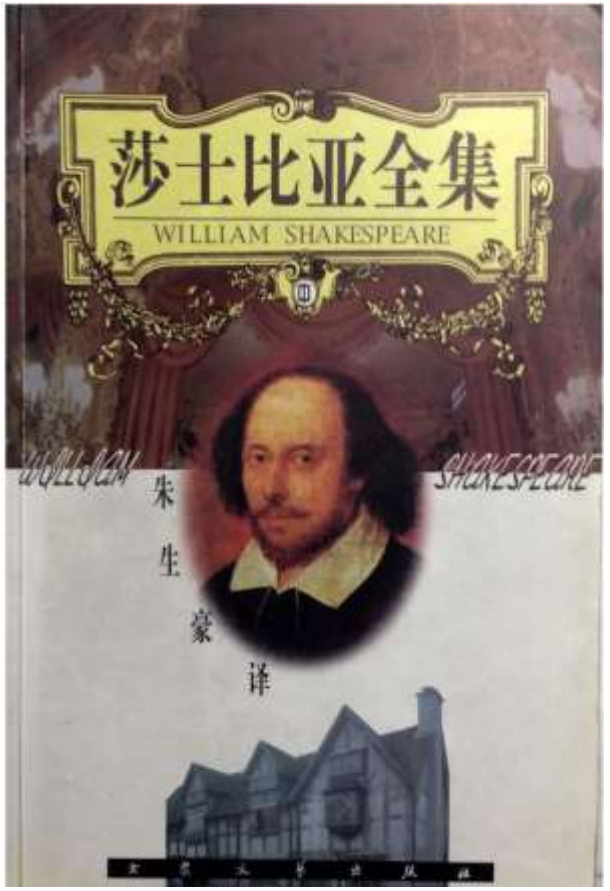
- 莎士比亚的哪部剧本包含Brutus及Caesar但是不包含Calpurnia?
- 布尔表达式为
- **Brutus AND Caesar AND NOT Calpurnia**

一个简单的例子(《莎士比亚全集》)

- **笨方法：从头到尾扫描所有剧本，对每部剧本判断它是否包含**

  Brutus AND Caesar，同时又不包含Calpurnia





一个简单的例子(《莎士比亚全集》)

- **笨方法为什么不好?**
  - **速度超慢 (特别是大型文档集)**
  - **处理NOT Calpurnia 并不容易 (一旦包含即可停止判断)**
  - **不太容易支持其他操作 (e.g., find the word Romans near countrymen)**
  - **不支持检索结果的排序 (即只返回较好的结果)**

词项-文档(term-doc)的关联矩阵

- **词项 Term**
  - **索引的单位，通常可以用词来表示**
- **文档 doc**
  - **Document**
  - **检索系统的检索对象**
- **关联矩阵**
  - **即用一个矩阵来表示各个点和每条边之间的关系**
  - **词项-文档关系，term-doc**

词项-文档(term-doc)的关联矩阵

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Brutus AND **Caesar** BUT NOT **Calpurnia**

若某剧本包含某单词，则该位置为1，否则为0

关联向量(incidence vectors)

- 关联矩阵的每一列都是 0/1向量，每个0/1都对应一个词项
- 给定查询 **Brutus AND Caesar AND NOT Calpurnia**
- 取出三个列向量，并对Calpurnia 的列向量求反，最后按位进行与操作

- 110100 AND 110111 AND 101111 = 100100

上述查询的结果文档

- 1. Antony and Cleopatra, Act III, Scene ii
  - Agrippa [Aside to DOMITIUS ENOBARBUS]: Why, Enobarbus,
  - When Antony found Julius Caesar dead,
  - He cried almost to roaring; and he wept
  - When at Philippi he found Brutus slain.
- 4. Hamlet, Act III, Scene ii
  - Lord Polonius: I did enact Julius Caesar I was killed i' the
  - Capitol; Brutus killed me.



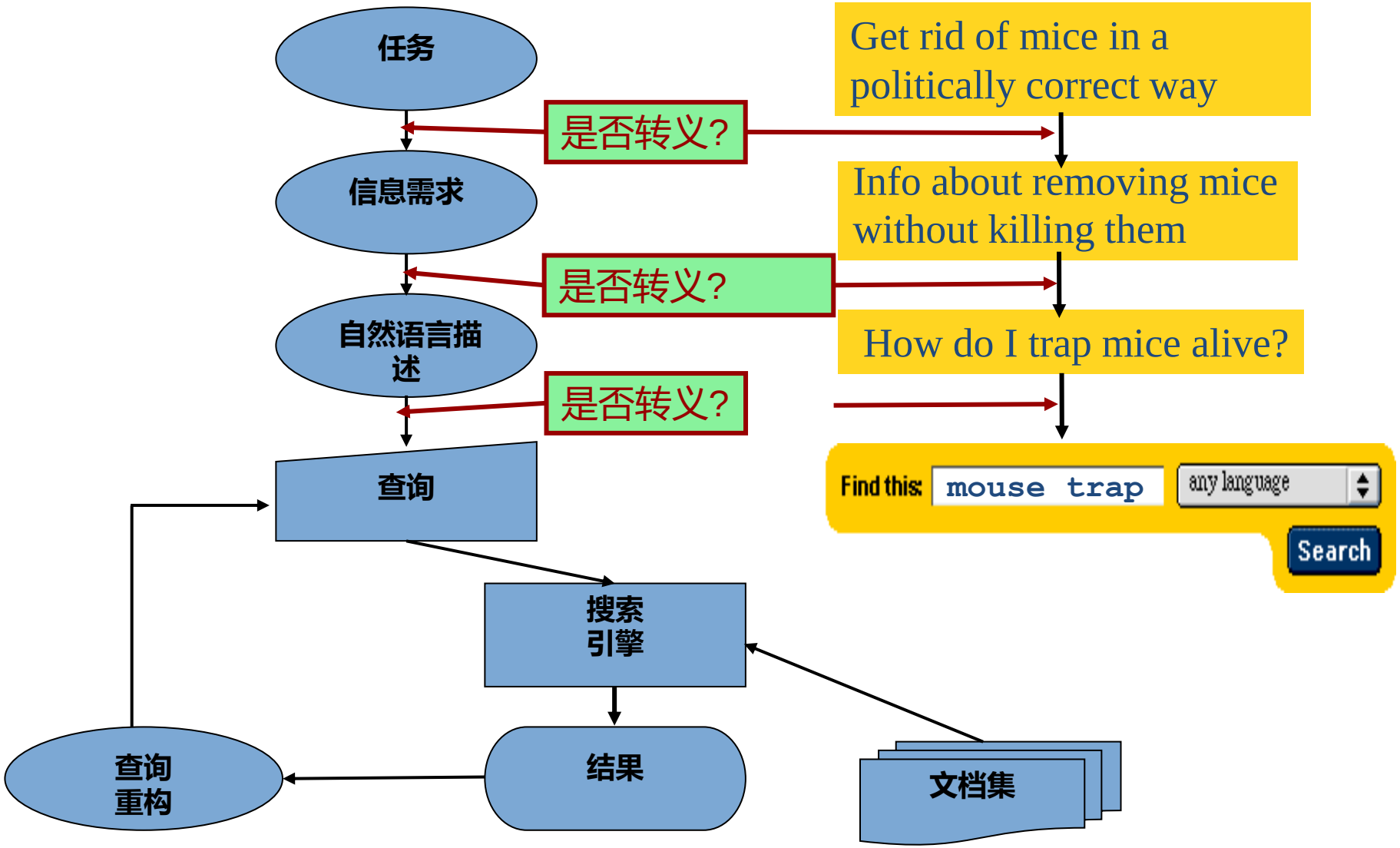
上述查询的结果文档


Microsoft 网上红心大战
游戏(G) 帮助(H)
Pauline
Michele
Ben
Microsoft 网上红心大战名言
亲爱的朋友们，其实我并不是
要偷走你们的心...
- 恺撒大帝，第三场，第二幕
确定
向左传
10 J 2 4 K A 2 5 6 7 8 K A
请选三张牌传给 Pauline。

IR中的基本假设

- **文档集 Collection: 由固定数目的文档组成**
- **目标: 返回与用户需求相关的文档并辅助用户来完成某项任务**
- **相关性 Relevance**
  - **主观的概念**
  - **反映对象的匹配程度**
  - **不同应用相关性不同**

典型的搜索过程


任务
是否转义?
Get rid of mice in a politically correct way
信息需求
是否转义?
Info about removing mice without killing them
自然语言描述
是否转义?
How do I trap mice alive?
查询
Find this:
mouse trap
any language
Search
搜索引擎
查询重构
结果
文档集

检索效果的评价指标

- **最重要的两个指标**
  - **正确率----Precision**
  - **召回率----Recall**

检索效果的评价指标

- **正确率(Precision)**
  - **返回结果文档中正确的比例**
  - 如返回80篇文档，其中20篇相关，正确率1/4
- **召回率(Recall)**
  - **全部相关文档中被返回的比例**
  - 如返回80篇文档，其中20篇相关，但是总相关文档100篇，召回率1/5

检索效果的评价指标

- **正确率和召回率反映检索效果的两个方面，缺一不可**
  - **只返回一个非常可靠的结果，正确率100%，召回率低**
  - **全部返回，正确率低，召回率100%**

大文档集

- 假定N = 1 百万篇文档(1M), 每篇有1000个词(1K)
- 假定每个词平均有6个字节(包括空格和标点符号)
- **试计算：所有文档占用的空间大小**
- **答：所有文档将约占6GB 空间**

词项-文档矩阵将非常大

- **假定词汇表的大小(即词项个数) M = 500K**
- **矩阵大小为 500K x 1M=500G**
- **每篇文档的平均长度为1000个单词**
- **该矩阵中最多有10亿(1G)个1** ← Why?
- **计算词项-文档矩阵中元素1的比例**
  - 词项-文档矩阵高度稀疏(sparse)
  - 稀疏矩阵
- **更好的表示方式--比如仅记录所有1的位置**

稀疏矩阵

- 如果在矩阵中，多数的元素为0，称此矩阵为

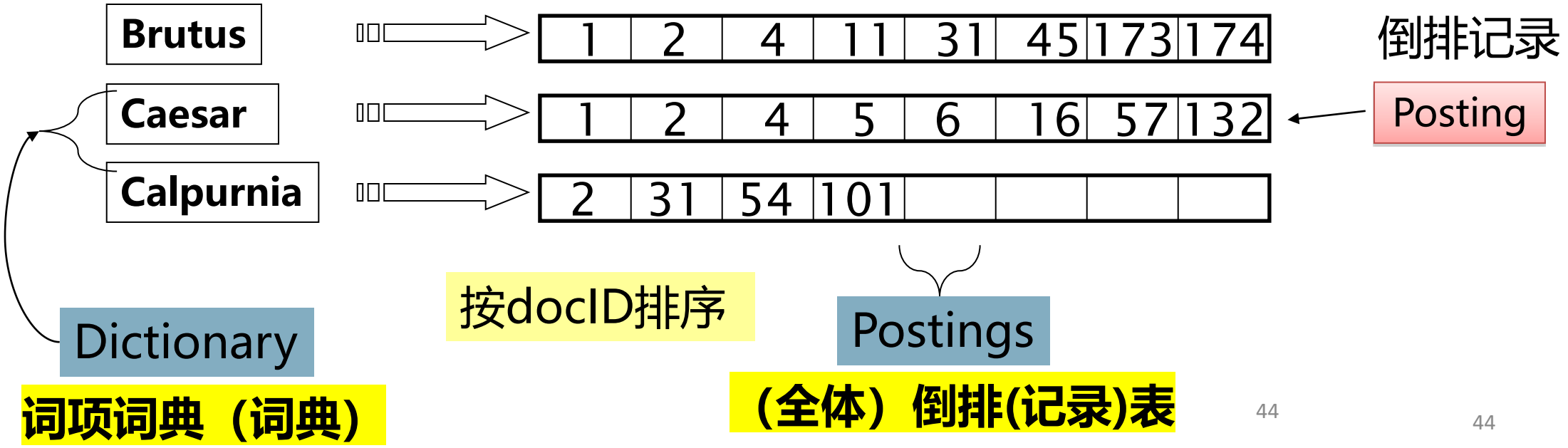
- **稀疏矩阵（sparse matrix）**

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 5 \\ 3 & 0 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 & 0 & 22 & 0 & 0 & 15 \\ 0 & 11 & 0 & 0 & 0 & 17 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 39 & 0 \\ 91 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 28 & 0 & 0 & 0 & 0 \end{pmatrix}$$

倒排索引(Inverted index)

- 左部称为**词项词典**（dictionary，简称词典，有时也称为vocabulary）
- **dictionary 指下图的数据结构，而vocabulary则指词汇表**
- 每个词项都有一个记录出现该词项的所有文档的列表，该表中的每个元素记录的是词项在某文档中的一次出现信息（该信息中往往还包括**词项在文档中出现的位置**）



44

倒排索引(Inverted index)

- 表中的每个元素通常称为**倒排记录（posting）**
- 每个词项对应的整个表称为**倒排记录表**（posting list）或**倒排表**（inverted list）
- 所有词项的倒排记录表一起构成**全体倒排记录表**（postings）
- 下图的词典按照**字母顺序**进行排序，而倒排记录表则按照**文档ID号**进行排序



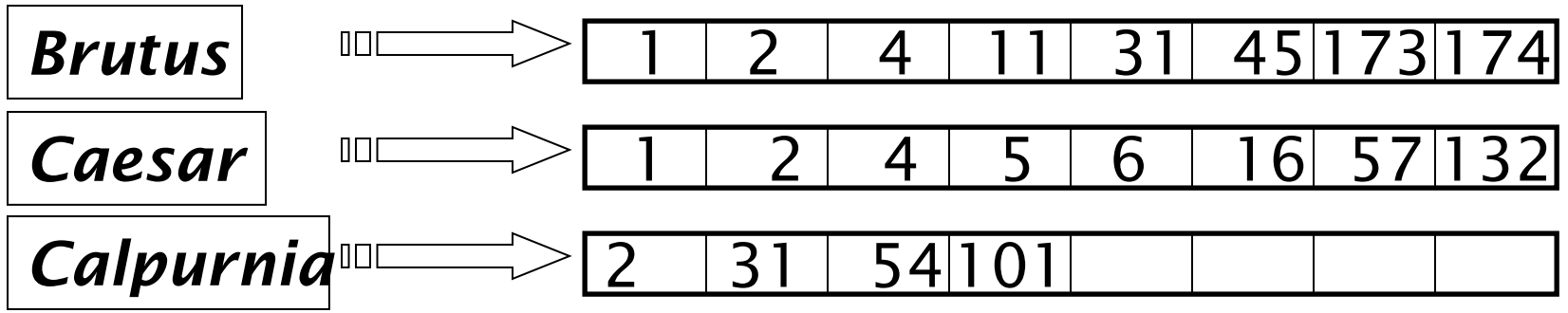
45

倒排索引(Inverted index)

- **Inverted index**
- 一个未经处理的数据库中，一般是以文档ID作为**索引**，以文档内容作为**记录**
- Inverted index 指的是将单词或记录作为**索引**，将文档ID作为**记录**，通过单词或记录查找到其所在的**文档**

倒排索引(Inverted index)

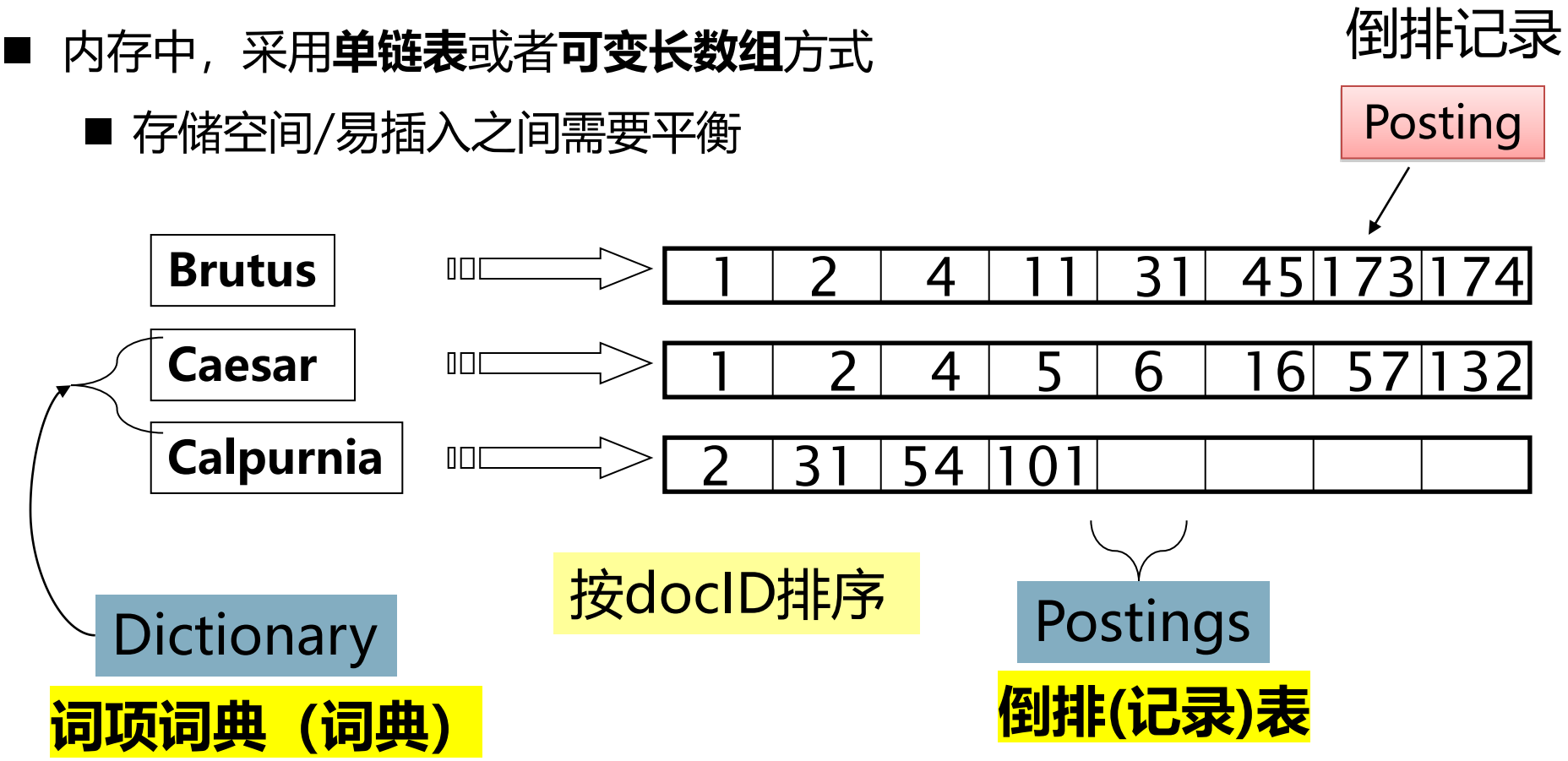
- 对每个词项 t , 记录所有包含 t 的文档列表.
  - 每篇文档用一个唯一的 **docID**来表示，通常是正整数，如1,2,3…
- 能否采用**定长数组**的方式来存储**docID**列表?



文档14中加入单词*Caesar*时该如何处理?

倒排索引(Inverted index)

- 通常采用**变长表**方式
  - 磁盘上，**顺序存储**方式比较好，便于快速读取
  - 内存中，采用**单链表**或者**可变长数组**方式
    - 存储空间/易插入之间需要平衡



倒排记录表在内存中的存储方式

- 内存中，采用**单链表**或者**可变长数组**方式
- **单链表**
  - **便于文档的插入和更新**
  - **增加指针**
- **可变长数组**
  - **节省指针消耗的空间**
  - **采用连续内存存储，充分利用计算机缓存（cache）技术来提高访问速度**

倒排索引构建



待索引文档

Friends, Romans, countrymen.

Tokenizer

词条化工具

词条流

Friends | Romans | Countrymen

More on these later.

Linguistic modules

语言分析工具

修改后的词条

friend | roman | countryman

倒排索引

Indexer

friend	2 → 4
roman	1 → 2
countryman	13 → 16

索引构建过程: 词条序列

- <词条，docID>二元组

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

索引构建过程: 排序

1. **按词项排序**
2. **每个词项按docID排序**

索引构建的核心步骤

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

索引构建过程: 词典 & 倒排记录表

- **某个词项在单篇文档中的多次出现会被合并**
- 拆分成**词典和倒排记录表**两部分
- 每个词项出现的**文档数目**

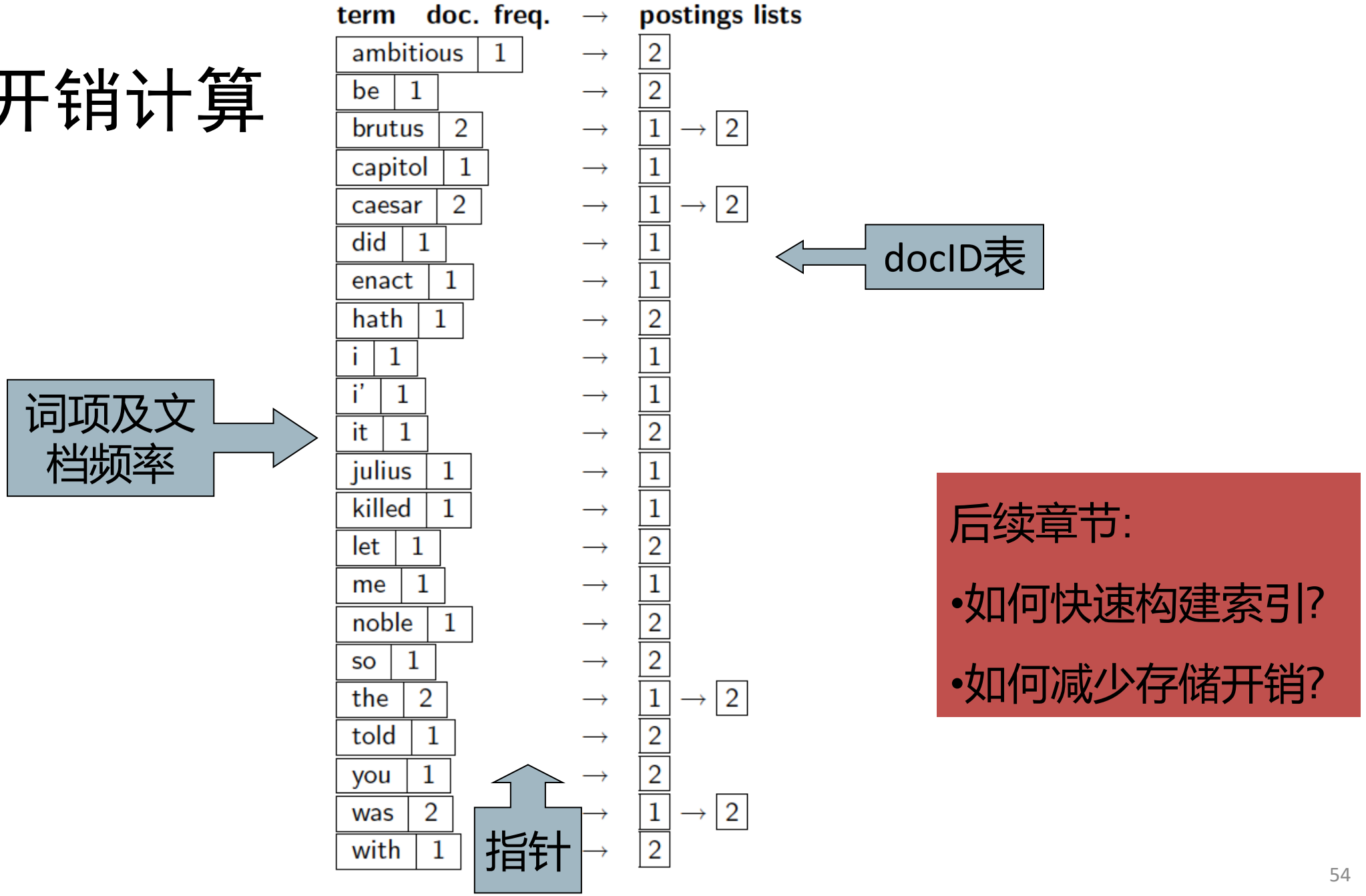
(doc. frequency, DF)会被加入

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

存储开销计算



后续章节:

- 如何快速构建索引?
- 如何减少存储开销?

Exercise

- **The main two parts of index construction are _____and_____.**
- **dictionary and inverted posting lists**

- 索引构建的主要两部分分别是_____和_____。
- 词典　倒排记录表

提纲

① 信息检索概述

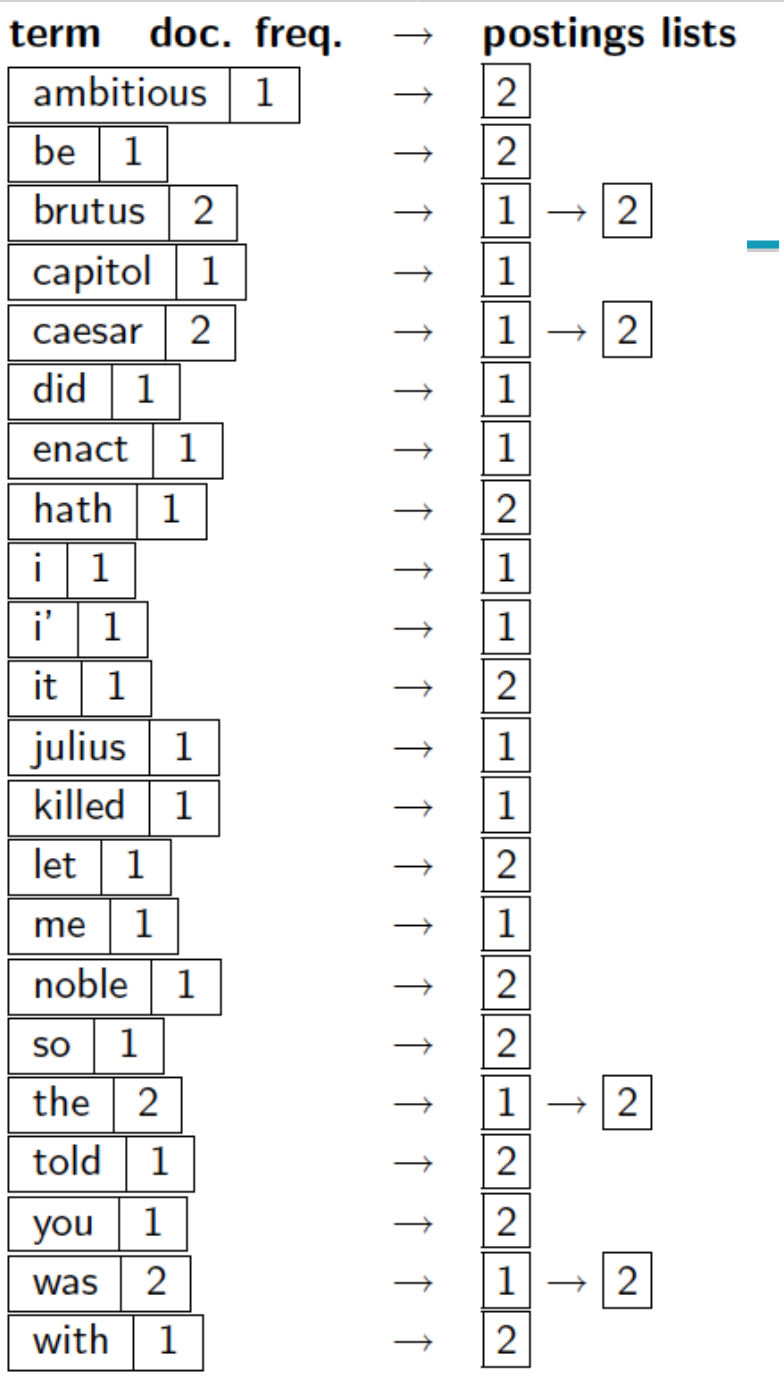
② 倒排索引

③ 布尔查询的处理

假定索引已经构建好

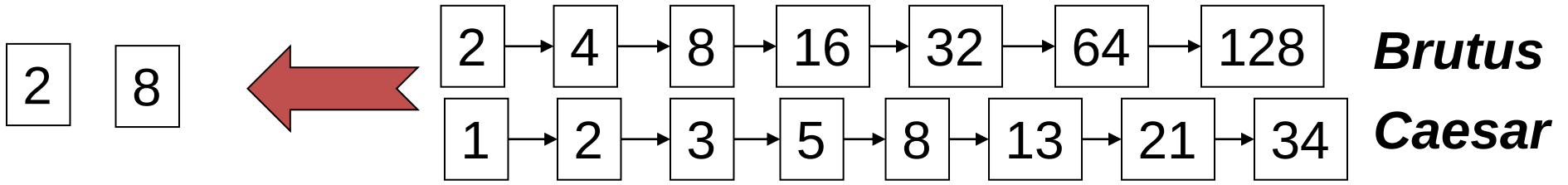
- 如何利用该**索引**来处理查询?
- 如何处理**不同类型**的查询?
  - 比如带通配符的查询 “**信息*检索**”

主要内容



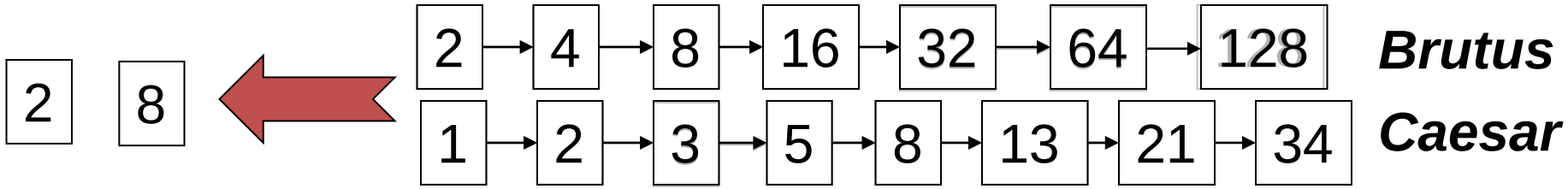
AND查询的处理

- 考虑如下查询（从简单的布尔表达式入手）
  - **Brutus AND Caesar**
  - 在词典中定位 Brutus
    - 返回**对应倒排记录表(对应的docID)**
  - 在词典中定位Caesar
    - 再返回对应倒排记录表
  - **合并(Merge)**两个倒排记录表，即求交集（**逻辑“与”操作**）



合并过程

- **每个倒排记录表都有一个定位指针，两个指针同时从前往后扫描，每次比较当前指针对应倒排记录，然后移动某个或两个指针。合并时间为两个表长之和的线性时间**



- 假定表长分别为x和y，求交集的过程需要为 O(x+y)次操作
- 上述合并算法的时间复杂度为 Θ(N)，N为文档数目
- **关键原因: 倒排记录表按照docID排序**

上述合并算法的伪代码描述

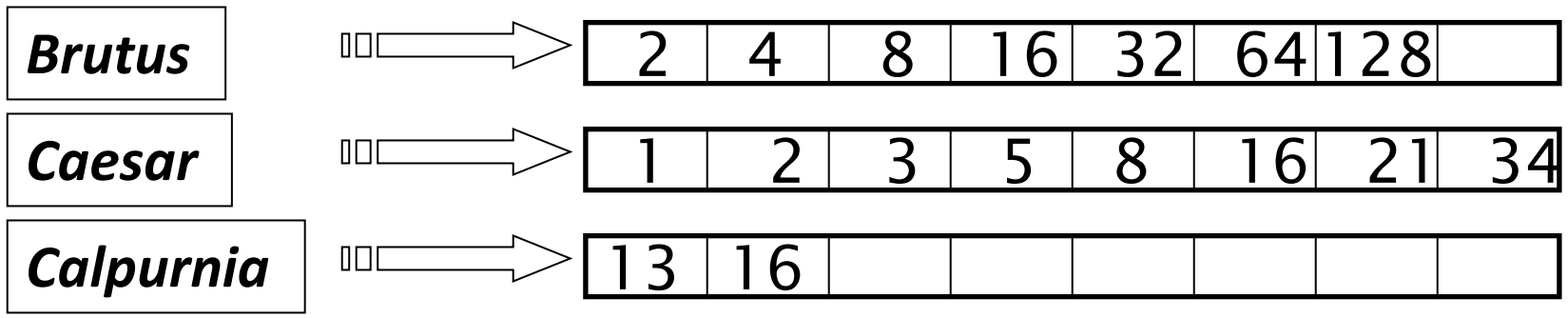
```
INTERSECT(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then p1 ← next(p1)
 9                else p2 ← next(p2)
10  return answer
```

其它布尔查询的处理

- **OR表达式：Brutus OR Caesar**
  - **两个倒排记录表的并集**
- **NOT表达式： Brutus AND NOT Caesar**
  - **两个倒排记录表的减**
- **可以对上述合并算法进行扩展，使之能够用于处理更复杂的查询**
  - **(Brutus OR Caesar) AND NOT (Antony OR Cleopatra)**
- **查询处理的效率问题！**

查询优化

- 查询优化（query optimization）指的是如何通过组织查询的处理过程来使处理工作量最小
- 对布尔查询进行优化要考虑的一个主要因素是倒排记录表的访问顺序
- 考虑 n 个词项的 AND，对每个词项，取出其倒排记录表，然后两两合并

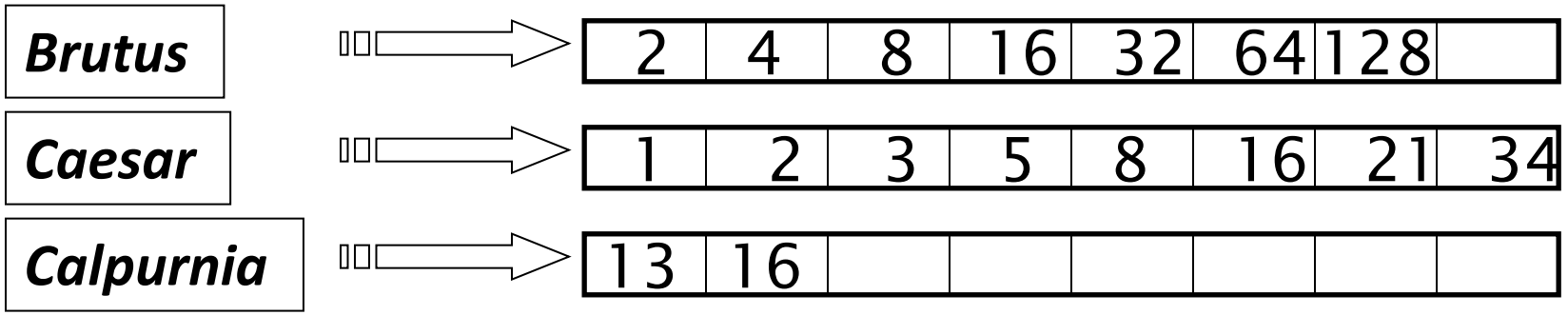


查询: ***Brutus*** *AND* ***Calpurnia*** *AND* ***Caesar***

查询优化

- **如果先合并两个最短的倒排记录表，那么所有中间结果的大小都不会超过最短的倒排记录表**
- **按照表从小到大(即df从小到大)的顺序进行处理**
  - **每次从最小的开始合并**

这是为什么保存df的原因之一



相当于处理查询 (*Calpurnia AND Brutus) AND Caesar*

更通用的优化策略

- e.g., (madding OR crowd) AND (ignoble OR strife)
  - 每个布尔表达式都能转换成上述形式(合取范式)
- **获得每个词项的df**
- 通过将词项的df相加，估计每个OR表达式对应的倒排记录表的大小
- 按照上述估计**从小到大依次处理**每个OR表达式

更通用的优化策略

- 对于任意的布尔查询，我们必须计算并临时**保存中间表达式**的结果
- 但是在很多情况下，不论是由于查询语言本身的性质所决定，还是仅仅由于这是用户所提交的最普遍的查询类型，查询往往是由纯**“与”操作**构成的
- 在这种情况下，不是将倒排记录表合并看成两个输入加一个不同输出的函数，而是将**每个返回的倒排记录表和当前内存的中间结果进行合并**，这样做效率更高
- 最初的中间结果中可以调入**最小文档频率的词项所对应的倒排记录表**

布尔检索的优点

- **构建简单**
  - 或许是构建IR系统的一种最简单方式
- **在30多年中是最主要的检索工具**
- **当前许多搜索系统仍然使用布尔检索模型**
    - 电子邮件
    - 文献编目（知网）
    - Mac OS X Spotlight工具

对基本布尔操作的扩展及有序检索

- **和布尔检索模型相对的是排序检索模型或有序检索模型（ranked retrieval model）**
- 后者不是通过具有精确语义的逻辑表达式来构建查询，而往往是采用一个或者多个词来构成**自由文本查询（free text query）**
- 有序检索系统要能够确定哪篇文档最能满足用户的需求
- 尽管**有序检索模型**在学术界已经提出并发展了几十年，也有很多优点，但是在上世纪90年代初（差不多万维网开始出现的时间）之前，在大型的商业信息提供商30年来所提供的检索系统中，**基于布尔模型的检索系统还一直占主流地位，有时甚至是检索系统中唯一的检索模型**

对基本布尔操作的扩展及有序检索

- **这些检索系统中并不只包含我们迄今为止提到的基本的布尔运算(AND、OR 及NOT)**
- 严格的布尔运算所得到的无序结果集远远不能使用户满意，所以，这些系统中往往加入了更多的操作，如“**词项邻近**”（**term proximity**）操作
- **邻近操作符（proximity）**用于指定查询中的两个词项应该在文档中互相靠近，靠近程度通常采用两者之间的词的个数或者是否同在某个结构单元（如句子或段落）中出现来衡量
  - 例子：**论文重复率检测，一种标准是16个连续字符重复**
  - 扩展：知识的意义

布尔检索例子: WestLaw　http://www.westlaw.com/

- 世界上最大的**商业化法律搜索服务引擎** (from 付费用户数目)
  - (1975年开始提供服务; 1992年加入排序功能)
  - 几十T数据，700,000用户
  - 每天大约有50 万个用户对数十T字节的文本数据进行数百万次的查询
- 大部分用户仍然使用**布尔查询**
  - 尽管在1992 年面向有序检索模型的自由文本查询已经添加到搜索系统中（Westlaw 称这种搜索为**Natural Language**），但是使用布尔搜索的用户的比例仍然更高

布尔检索例子: WestLaw　http://www.westlaw.com/

- **查询的例子: 有关对政府侵权行为进行索赔的诉讼时效**
- What is the statute of limitations in cases involving the federal tort claims act?
- **LIMIT! /3 STATUTE ACTION /S FEDERAL /2 TORT /3 CLAIM**
- **/3 = within 3 words, /S = in same sentence**
- **空格表示逻辑“或”关系（OR），&表示逻辑“与”关系（AND）**
- **/s、/p、/k 分别表示处于同一句子、段落和k个词之内**

布尔检索例子: WestLaw　http://www.westlaw.com/

- **另一个例子: 残疾人士能够进入工作场所的要求**
- (Requirements for disabled people to be able to access a workplace)
- **disabl! /p access! /s work-site work-place (employment /3 place)**
- **双引号表示短语查询**
- **感叹号!表示尾通配符查询**

练习

- **利用Westlaw 系统的语法构造一个查询：**
  - **通过它可以找到professor、 teacher 或lecturer中的任意一个词**
  - **并且该词和动词explain 在一个句子中出现，其中explain以某种形式出现。**

布尔检索例子: WestLaw　http://www.westlaw.com/

- 扩展的布尔操作符
- 很多**专业人士**喜欢使用**布尔搜索**
  - 非常清楚想要查什么、能得到什么
- 但是这并不意味着布尔搜索其实际效果就很好….
- **布尔搜索的普遍问题**
  - **采用 AND 操作符产生的结果正确率高但是召回率偏低**
  - **采用 OR 操作符召回率高但是正确率低**

Google支持布尔查询

- 例1：想查关于2011年快女 6进5 比赛的新闻，用布尔表达式怎么构造查询?
- (2011 OR 今年) AND (快乐女声 OR 快女 OR 快乐女生) AND (6进5 OR 六进五 OR 六 AND 进 AND 五)
- 例2：2023年成都大运会男篮四分之一决赛
  - 2023: 二零二三 OR 今年；　　男篮：男子篮球
  - 大运会：大学生运动会
- 表达式相当复杂，构造困难!
- 不严格的话结果过多，而且很多不相关；非常严格的话结果会很少，漏掉很多结果

布尔检索的缺点

- **布尔查询构建复杂，不适合普通用户**
- 构建不当，检索结果过多或者过少
- 程序员笑话
  - “晚上回来买一个西瓜，如果看到苹果，就买两个”

布尔检索的缺点

- **没有充分利用词项的频率信息**
  - 1 vs. 0 次出现
  - 2 vs. 1次出现
  - 3 vs. 2次出现, …
  - 通常出现的越多越好，需要利用词项在文档中的**词项频率(term frequency, tf)信息**
- **不能对检索结果进行排序**

参考资料

- **《信息检索导论》，第一章**
- **莎士比亚全集:**
  - **http://www.rhymezone.com/shakespeare/**
- **Managing Gigabytes(深入搜索引擎), 3.2节**
- **《现代信息检索》, 8.2节**

Extra

- 网络安全意识
- 计算思维

- “爬虫”
- 相对于通用网络爬虫，聚焦爬虫还需要解决三个主要问题：
- （1）对抓取目标的描述或定义
- （2）对网页或数据的分析与过滤
- （3）对URL的搜索策略

“爬虫” Web crawler





Review

信息检索 Information Retrieval

- **信息检索**是从大规模非结构化数据（通常是**文本**）
- 的集合（通常保存在计算机上）中
- 找出满足用户信息需求的资料（通常是**文档**）
- 的过程

信息检索 Information Retrieval

- Document – 文档
  - 检索系统的检索对象
- Unstructured – 非结构化
  - 没有清晰和明显的语义结构的数据，计算机不易处理
- Information need – 信息需求
- Collection – 文档集、语料库

布尔检索

- **针对布尔查询的检索，布尔查询是指利用 AND, OR 或者 NOT操作符将词项连接起来的查询**
  - 信息 AND 检索
  - 信息 OR 检索
  - 信息 AND 检索 AND NOT 教材
- Google的高级搜索?
- Baidu?

词项-文档(term-doc)的关联矩阵

- **词项 Term**
  - **索引的单位**，通常可以用词来表示
- **文档 Doc**
  - Document
  - 检索系统的检索对象
- **关联矩阵**
  - 即用一个矩阵来表示各个点和每条边之间的关系
  - 词项-文档关系

词项-文档(term-doc)的关联矩阵

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Brutus AND **Caesar** BUT NOT **Calpurnia**

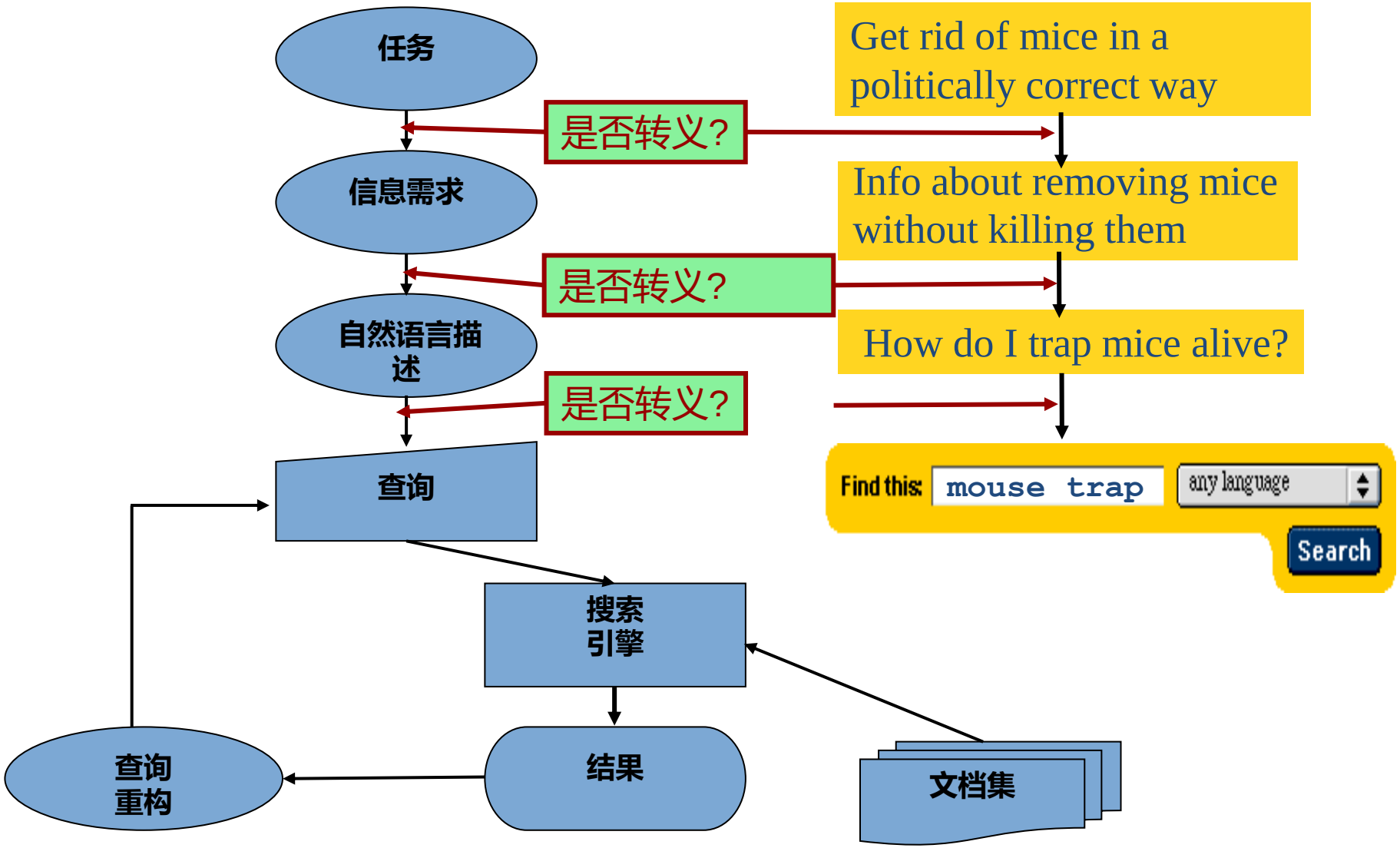
若某剧本包含某单词，则该位置为1，否则为0

关联向量(incidence vectors)

- 关联矩阵的每一列都是 0/1向量，每个0/1都对应一个词项
- 给定查询 **Brutus AND Caesar AND NOT Calpurnia**
- 取出三个列向量，并对Calpurnia 的列向量求反，最后按位进行与操作

- 110100 AND 110111 AND 101111 = 100100

典型的搜索过程


任务
是否转义?
Get rid of mice in a politically correct way
信息需求
是否转义?
Info about removing mice without killing them
自然语言描述
是否转义?
How do I trap mice alive?
查询
Find this:
mouse trap
any language
Search
搜索引擎
查询重构
结果
文档集

检索效果的评价指标

- **正确率(Precision)**
  - **返回结果文档中正确的比例**
  - 如返回80篇文档，其中20篇相关，正确率1/4
- **召回率(Recall)**
  - **全部相关文档中被返回的比例**
  - 如返回80篇文档，其中20篇相关，但是总相关文档100篇，召回率1/5